



Text-to-SQL Dashboard

Complete Documentation

Ask your database questions in plain English and get back validated, read-only SQL, a results table, and an auto-picked chart — powered by a fully local LLM stack, with layered input safety, rate limiting, and a documented governance process behind it.

 Ollama (local LLM) ·  LangGraph ·  ChromaDB ·  SQLAlchemy ·  sqlglot validation · 
Streamlit + Plotly

Covers how the application works, how to set it up and configure it, how to use it day to day, and how its risk posture is documented. For a deep, code-level technical walkthrough, see [docs/ARCHITECTURE.md](#).

CONTENTS

Table of Contents

1. Overview & Key Features
2. How It Works
3. Project Structure
4. Getting Started (Install & Configure)
5. Using the Dashboard
6. Under the Hood: Retrieval, Retries & Safety
7. Input Safety: Rejections, Rate Limits & Cost Guards
8. Security & Data Privacy
9. Governance & Risk Posture
10. Testing & Quality Checks
11. Known Limitations
12. Troubleshooting
13. Frequently Asked Questions
14. Glossary
15. Further Reading

Overview & Key Features

What this app does

This dashboard lets you ask questions about a real, live database in plain English — no SQL required. You type a question, the app writes a SQL query for you, shows it to you for review, and only ever runs it once you approve. Results appear as a table, with a chart drawn automatically when the data shape supports one, and an optional plain-English summary of what the result shows.

Every query is read-only. The app can look at your data but is never allowed to change it — enforced in code, not just by convention (see Chapter 8). Every typed question is also screened before it ever reaches the AI model, so obviously off-topic or manipulative input is rejected cleanly instead of silently wasting a generation cycle (Chapter 7).

A typical flow

1. **Ask** — Type a question, e.g. "Who are the top 5 customers by total sales?"
2. **Screen** — The question is checked for length, obvious manipulation attempts, and whether it's even database-shaped, before anything expensive happens.
3. **Generate** — The app finds the relevant tables and writes a SQL query, self-correcting if its first attempt has a mistake.
4. **Review** — You see the exact SQL (and can hand-edit it) before anything touches the database. If the query looks unusually broad, you're warned before you wait, not after.
5. **Confirm & Run** — Results appear as a table, with an optional auto-picked chart and a grounded, plain-English insight.

Who it's for

Anyone who needs answers from a database but doesn't write SQL day to day — analysts, product managers, support staff — as well as engineers who want a fast, inspectable way to explore a schema they don't already have memorized.

Key features

Feature	What it means for you
Self-correcting answers	If the first SQL attempt has a mistake, the app automatically tries again using the specific error — up to a capped number of attempts — instead of just failing.
Read-only by construction	Every query is checked against an allowlist that accepts only SELECT-style statements. Nothing your questions do can insert, update, delete, or alter

	data.
Schema-aware, not schema-dumping	The AI model never sees your entire database schema at once. It's shown only the handful of tables relevant to your specific question.
Works with your real database	PostgreSQL, MySQL, SQL Server, or Oracle — configured entirely through a settings file, with no sample or hardcoded data anywhere.
Human-readable results	Internal ID/key columns are hidden from the default view, and column names are expanded into plain language, without ever changing the underlying query.
Curated business context	A hand-maintained notes file describes what each table means in plain language — read fresh on every question, so a correction takes effect immediately.
Fully local AI	The language model runs on your own machine via Ollama. No API keys, no per-question cost, and no data ever leaves the machine.
Transparent retries	Every attempt the app makes, successful or not, is recorded and viewable in a "Retry timeline" — nothing about the self-correction process is hidden.
 Input safety screening	Every question is checked for prompt-injection phrasing (including disguised/homoglyph attempts), off-topic requests, and length — before any AI call is made.
 Rate limiting	Two independent, lightweight limits guard against accidental or deliberate overload — a per-session cap on questions and a stricter cap on AI generation calls.
 Proactive cost warnings	Before an unusually broad query runs, its estimated size is checked. Moderately large queries get a "this may take a moment" heads-up; extremely large ones are stopped before they run at all.
 Follow-up questions	You can ask a follow-up like "now break that down by quarter" or "just show the top 3 of those" — the app recognizes it refers to your previous question and resolves it in context.
 AI insight	After a multi-row result, an optional one-sentence, plain-English summary appears above the table — every number in it is checked against the actual result before it's shown, so it can't state a figure the data doesn't support.
Built-in accuracy checks	A growable library of real test questions with pass/fail checks — including a dedicated category of adversarial/malicious inputs the app must correctly refuse — catches regressions before they ship.

Why it works this way: most "chat with your database" tools either trust the AI's SQL blindly, or hide its reasoning behind an opaque black box. This app treats AI output — and everything that reaches the AI, including your question and the database's own content — as untrusted by construction. Every query is parsed and checked before it can run, every retry is visible, and every rejection has a clear, specific reason.

How It Works

Under the hood, the app is a small, explicit state machine rather than a free-form AI agent that decides its own next move. Every possible path — including every retry, rejection, and follow-up resolution — is a fixed, readable step in a diagram, not something that emerges unpredictably from the model's own reasoning.

1. **Sanitize & Screen** — The raw question is normalized and checked for length, injection-style phrasing, and obvious off-topic requests. A clean rejection happens here before anything else runs (Chapter 7).
2. **Classify Follow-up** — Is this a standalone question, a follow-up to the previous one, or too ambiguous to resolve without asking you to clarify?
3. **Retrieve Schema** — Finds the handful of tables most relevant to your question (ChromaDB search, Chapter 6).
4. **Generate SQL** — The local AI model (Ollama) writes a candidate query using only those tables, guarded by a rate limit on generation calls.
5. **Validate SQL** — The query is parsed and checked: is it a single, genuine read-only SELECT? Anything else is rejected before it can run.
6. **Estimate Cost** — Before execution, the query's likely size is estimated without running it. An extremely broad query is stopped here and fed back as a correctable mistake.
7. **Execute** — The validated query runs against a read-only connection, with a row limit and a timeout.
8. **Generate Insight** (optional) — A short, grounded plain-English summary of the result, if the toggle is on and the result is substantial enough to be worth summarizing.

If a step fails in a way that's worth retrying, the app loops back to an earlier step with the specific error in hand — up to a configurable limit — rather than giving up on the first mistake. Chapter 6 explains exactly which failures get retried and which don't.

The technology underneath

Layer	Choice	Why this one
AI / LLM runtime	Ollama (default model <code>llama3.1:8b</code>)	Runs entirely on your machine — no API keys, no data leaving the network, no per-question cost.
Orchestration	LangGraph	An explicit, inspectable state machine instead of a free-form agent loop.
Schema retrieval	ChromaDB	A local vector search index that finds the small set of relevant tables instead of dumping the whole schema into every question.

Database access	SQLAlchemy	One consistent connection layer across four supported databases, driven entirely by configuration.
SQL safety check	sqlglot	Parses the actual query structure and allowlists what's permitted, rather than scanning for "bad words" in the text.
Interface	Streamlit + Plotly	A fast, reviewable web UI — editable SQL box, retry timeline, schema browser — without a separate frontend build.

In one sentence: your question is screened before it talks to the AI, the AI only sees a relevant slice of the schema, its SQL is independently verified and cost-checked before it can run, and every step of that journey — including every rejection and retry — is visible to you in the UI.

Project Structure

You don't need to read code to use the app, but this map is useful if you're curious where a particular behavior lives, or if you're evaluating the project technically.

Folder	What lives here
agent/	The AI pipeline: the graph (<code>graph.py</code> , <code>nodes.py</code>), the SQL safety allowlist (<code>sql_validator.py</code>), input screening (<code>input_guard.py</code>), follow-up classification (<code>followup.py</code>), rate limiting (<code>rate_limit.py</code>), grounded-insight logic (<code>insight.py</code>), and error classification for the retry logic.
security/	 Shared input/output normalization (<code>sanitization.py</code>) — Unicode normalization, homoglyph handling, log-safe truncation — used by both the input screener and anything that puts database content into a prompt.
config/	All tunable settings in one place (<code>settings.py</code>) — model name, database connection fields, row limits, timeouts, rate limits, cost thresholds — plus the hand-maintained table-meaning notes (<code>table_descriptions.yaml</code>) and column-label rules.
db/	Database connection lifecycle, live schema introspection, sampled real values from coded columns, and proactive query-cost estimation (<code>query_cost.py</code>).
embeddings/	Builds and searches the ChromaDB index that powers schema-aware retrieval (Chapter 6).
ui/	The Streamlit web app (<code>app.py</code>), result-formatting helpers, and session/conversation history (<code>session_history.py</code>) that powers follow-up questions and the sidebar History panel.
scripts/	Standalone command-line tools: verify your database connection, build the schema index, and run the accuracy-eval harness.
tests/	The automated test suite — fully mocked, so it runs without a real database or Ollama — including a dedicated adversarial-input test module.
eval/	Real natural-language test questions with pass/fail expectations, including standalone questions, multi-turn follow-up sequences, adversarial inputs, and cost-estimation scenarios (Chapter 10).
docs/	This document, the deeper technical architecture write-up, and — new — the project's governance and change-control documentation (Chapter 9).

One rule worth knowing: nothing in this project hardcodes a database connection, host, or schema. Everything about your database — including which tables exist — is discovered live at startup and refreshed on demand. See Chapter 4 for how to point it at your own database.

Getting Started

Prerequisites

- Python 3.11 is the project's target version (see the note below if your machine only has a newer interpreter).
- Ollama, installed and running, with a model pulled — `ollama pull llama3.1:8b`.
- A database to connect to — PostgreSQL, MySQL, SQL Server, or Oracle. No sample database is bundled with the project by design.

Install (Windows / PowerShell)

```
git clone <this-repo-url>
cd Gen_AI_Project_TSQL
py -3.11 -m venv .venv          # or: python -m venv .venv
.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
pip install -r requirements.txt
Copy-Item .env.example .env
```

macOS/Linux/WSL users: the equivalent bash commands are in the README — the only difference is venv activation (`source .venv/bin/activate`) and using `cp` instead of `Copy-Item` .

Configure your database connection

Edit the new `.env` file with your real connection details. Every field is documented inline in `.env.example` ; the ones you'll always set:

Setting	Default	Meaning
DB_TYPE	—	One of <code>postgresql</code> , <code>mysql</code> , <code>mssql</code> , <code>oracle</code> .
DB_HOST	—	Server hostname or IP.
DB_PORT	engine default	5432 / 3306 / 1433 / 1521 respectively if left blank.
DB_NAME	—	Database (catalog) name.
DB_USER / DB_PASSWORD	—	Login credentials — use a dedicated read-only account (Chapter 8).
DB_SCHEMA	connection default	Optional — restrict what the app can see to one schema.

Settings introduced by the input-safety, rate-limiting, and cost-estimation layer

NEW These did not exist in earlier versions of this app — they're documented here for the first time.

Setting	Default	Meaning
MAX_QUESTION_LENGTH	500	Maximum accepted length (characters) of a typed question — rejected with a clear message rather than silently truncated.
QUESTION_RATE_LIMIT_PER_MINUTE	10	Max question submissions per minute, per browser session.
LLM_CALL_RATE_LIMIT_PER_MINUTE	20	Max AI generation calls per minute, across the whole app — stricter than the question limit, since one question's retries can multiply calls.
COST_ESTIMATION_ENABLED	true	Whether a non-executing cost check runs before a validated query executes.
COST_ESTIMATION_TIMEOUT_SECONDS	3	How long the cost check itself is allowed to take before being skipped.
COST_MODERATE_ROW_THRESHOLD	50,000	Estimated rows above which a query still runs, but you're warned first.
COST_HIGH_ROW_THRESHOLD	1,000,000	Estimated rows above which a query is not run at all — treated as a correctable mistake instead.
INSIGHT_MAX_TOKENS	120	Caps the length of the optional AI insight sentence.

Verify, build, and run

```
# 1. Confirm .env is correct before doing anything else
python scripts\test_db_connection.py

# 2. Introspect the schema and build the search index
python scripts\build_embeddings.py

# 3. Launch the app
streamlit run ui\app.py
```

PowerShell shortcuts for all of the above are in `tasks.ps1`. The app opens in your browser, normally at `http://localhost:8501`. If the database can't be reached, you'll see a clear "Database Connection Required" screen instead of a broken app.

Using the Dashboard

The sidebar

-  **Database Connection** — engine, database name, masked login, live Test Connection, Refresh Schema, and a Discovered Tables browser.
-  **Options** —  a "Generate AI insight" toggle, on by default, controlling whether a grounded plain-English summary is produced after a successful multi-row result.
-  **History** —  every question asked this session, with its outcome icon and timestamp. Session-only, cleared on refresh or restart.

Asking a question

Type your question into the chat box in plain English and press Enter. A status message describes what's happening — screening the input, retrieving schema, generating SQL, self-correcting if needed.

Asking a follow-up question

You don't have to restate the whole question each time. If your next message clearly refers back to the previous one — "now break that down by quarter," "just show the top 3 of those," "what about for 2013 instead?" — the app recognizes it as a follow-up and resolves it using your previous question and its result. When this happens, a small  Following up on: "..." note appears above the answer so it's always clear what context was used. If a message is too ambiguous to resolve on its own (no prior question in this session, or no clear subject), you'll be asked to clarify rather than have the app guess.

Reviewing the generated SQL

Before anything runs against your database, the app shows you the retrieved schema context and the generated SQL in an editable box. Nothing runs automatically — the SQL is only a proposal until you click **Confirm and Run**. Every click of that button — including after you've hand-edited the SQL — is independently re-validated before it's allowed to execute.

 If the query you're about to run is estimated to return an unusually large result, a short heads-up ( "this may take a moment") appears above the Confirm and Run button, before you wait for it — not after.

Confirm and Run

Click **Confirm and Run** to execute the query. Results appear below as a table.  If the result is substantial enough to be worth summarizing, an **AI insight** callout appears above the table with a one-sentence, plain-English observation (e.g. "Bikes accounted for roughly 60% of total sales") — every number in it is checked against the actual result data before display, so it never states a figure the data

doesn't support. Two checkboxes control how results are shown: "Show technical columns" and "Show chart."

Understanding the retry timeline

Sometimes the first SQL attempt doesn't work. When that happens, the app automatically tries again with the specific error in hand. Whenever more than one attempt happened, a **Retry timeline** section appears:

You'll see	What it means
 succeeded	This attempt worked; its results are what's shown.
 syntax error / missing reference	The SQL had a mistake or referenced the wrong table/column; the app corrected course and tried again.
 safety violation	The app blocked itself from running something other than a read-only lookup. This never retries — it's a hard stop, by design.
 timeout	The query was taking too long and was stopped. This also doesn't retry automatically.
  too expensive	The query's estimated size was far outside normal bounds — stopped before it ran, fed back as a correctable mistake so the model can narrow it.

When a question is declined outright

Some inputs never reach the AI model at all — an empty message, one that's too long, one that reads as an attempt to manipulate the app's instructions, or one that's clearly unrelated to your database (general trivia, "write me a poem," and similar). These are declined immediately with a clear, calm message rather than a generic error. See Chapter 7 for exactly what's screened and why.

If you're asking a lot of questions quickly

If you submit questions faster than the configured rate limit allows, you'll see a short, specific message telling you to slow down rather than a silent delay or a crash. This resets automatically within a minute.

Under the Hood: Retrieval, Retries & Safety

This chapter goes one level deeper than Chapter 2 for readers who want to understand why the app behaves the way it does. For the full code-level walkthrough, see docs/ARCHITECTURE.md.

Why the AI never sees the whole schema

A real production database can have hundreds of tables. Instead, each table's structure is pre-indexed via ChromaDB, and only the small number most relevant to your specific question — plus any tables needed to structurally connect them — are shown to the model.

Filling the gaps: connecting the dots between tables

The app looks at how the initially retrieved tables are related in the real schema and pulls in any missing connector tables needed to bridge them, so the model isn't left inventing a shortcut that doesn't exist.

Fixing "the column name lied to me"

The app samples a handful of real distinct values from qualifying columns (short, low-cardinality text columns only — never a key column, and never a column with enough distinct values to plausibly be personal information) and shows those real examples to the model.

The self-correction loop: not every failure is treated the same

What went wrong	Retries?	Why
Ordinary mistake in the SQL text	Yes	The model had the right information, it just wrote something slightly wrong.
Referenced a table/column that doesn't exist	Yes, differently	This can mean the wrong tables were retrieved in the first place, so the app goes back to the retrieval step.
Safety violation (attempted a non-read-only statement)	Never	A hard security stop — the app fails closed immediately, no matter how many retries remain.
Query timed out	Never	Retrying an expensive query with roughly the same shape wastes the whole retry budget on something a retry can't fix.
 Estimated cost far outside normal bounds	Yes	Same retry budget as an ordinary mistake — the estimate is fed back so the model can add a narrowing filter itself.

How "read-only" is actually enforced

1. **Structural check** — The generated SQL is parsed into its real structure and checked against an allowlist: only a single SELECT-shaped statement is ever accepted.
2. **Row cap** — Enforced two independent ways (a LIMIT added to the query itself, and a hard cap at fetch time).
3. **Timeout** — A long-running query is force-stopped rather than allowed to run indefinitely.

This check runs every time SQL is about to execute — including after you hand-edit the SQL box yourself. A hand-edit is treated exactly as untrustingly as anything the AI produced.

Input Safety: Rejections, Rate Limits & Cost Guards

This entire chapter documents functionality added in this update. It covers the checks that run *before* your question ever reaches the AI model, and the resource-protection checks that run around it.

What gets screened, and why

Every typed question passes through a screening step before anything else touches it:

Check	Catches	Rejection reason shown
Length cap	An absurdly long input, checked before any normalization work.	too_long
Emptiness	Blank or whitespace-only input.	empty
Injection-pattern detection	Phrasing that tries to redirect the AI's behavior — "ignore previous instructions," "you are now in developer mode," "reveal your system prompt," and disguised variants using look-alike characters from other alphabets (e.g. a Cyrillic letter standing in for a Latin one).	injection_detected
Off-topic / relevance check	Requests with nothing to do with your database — general trivia, "write me a poem," roleplay requests, or pure gibberish.	off_topic

Important nuance: this screening step is a fast, cheap first layer — deliberately *not* the app's actual security boundary. A sufficiently reworded manipulation attempt could get past it. What actually bounds the consequences if that happens is structural and sits deeper in the pipeline: the AI is explicitly instructed to treat your question (and anything read from the database) as data to convert, never as instructions to follow, and even a fully manipulated AI response can still only ever produce text — which the SQL validator then holds to a strict read-only allowlist before anything can execute. See Chapter 8.

Database content is screened the same way

It isn't just your typed question that's treated as untrusted. Table and column names, sampled real column values, and query result data shown in the AI insight all pass through the same Unicode normalization and control-character stripping before they're ever placed into a prompt. This matters because anyone who can write to the underlying database — not just someone typing into this app — could otherwise plant manipulation-style text inside ordinary-looking data (e.g. a product name designed to look like an instruction). Neutralizing it at the source means it can't influence what the AI generates.

Rate limiting

Two independent, deliberately simple limits — an in-memory counter, no external service, reset on every app restart:

Limit	Scope	Default
Question submissions	Per browser session	10 per minute
AI generation calls	Across the whole app	20 per minute

The generation-call limit is stricter and separately scoped because a single question's self-correction retries can call the AI model several times on their own — the question-level limit alone can't bound that.

Proactive query cost estimation

Before a validated query runs, the database's own query planner is asked to estimate its size — without actually running it:

Estimated size	What happens
Normal	Proceeds silently.
Moderate (≥ 50,000 rows by default)	Still runs, but you see a "this may take a moment" notice first.
High (≥ 1,000,000 rows by default)	Does not run at all — treated as a correctable mistake and fed back to the AI, same as a syntax error.

This check always "fails open": if the estimate itself errors out, times out, or isn't supported for your database engine, the query proceeds normally rather than being blocked by a broken check. It's a proactive early warning layered in front of the existing timeout, not a replacement for it.

Security & Data Privacy

What's actually guaranteed

- Generated SQL is restricted to a single read-only lookup via a structural (not keyword-based) allowlist, checked on every attempt, retry, and hand-edit.
- Query execution is row-capped and time-limited, enforced independently at two layers.
-  Every question is screened for injection-style phrasing and off-topic intent before it reaches the AI model — see Chapter 7.
-  Two independent rate limits guard against accidental and deliberate resource exhaustion.
-  An unusually broad query is flagged or blocked, before execution, based on a non-executing cost estimate.
- The app never logs a connection string, password, or full result row — and  rejected/flagged input is logged only as a short, length-capped excerpt, never in full.
- The AI model runs entirely on your own machine. No schema, question, or result data is ever sent to a hosted API.

What is not guaranteed

- The validator does not physically strip write privileges from the database connection itself. The real guarantee has two layers: the SQL allowlist, and the database login configured in `.env` being a genuinely read-only account.
-  The input-screening layer in Chapter 7 is a first filter, not the security boundary — see the callout there for what actually bounds the risk if it's bypassed.
- This project has not been through an independent security review, penetration test, or adversarial prompt-injection testing by anyone other than the author.
- It is not designed for multi-tenant or production deployment — no authentication, no per-user authorization.  Rate limiting exists now, but it's a basic, in-memory, single-process safeguard sized for one local user, not a substitute for real multi-tenant rate limiting.
-  Column-level sensitivity classification (marking specific columns as restricted and enforcing that automatically) is **not yet built** — see Chapter 9's Governance chapter for how this gap is tracked.

Bottom line: do not point this at a production database, or a database containing real sensitive/personal data, without an independent security review and your own risk assessment first. For local exploration, demos, or a sandboxed/synthetic dataset, this is a reasonable posture. For anything more sensitive, treat this as a starting point, not a finished answer.

Setting up a read-only database account

Create a dedicated database login with SELECT-only privileges (no INSERT/UPDATE/DELETE/DDL) and point `DB_USER` / `DB_PASSWORD` in `.env` at it. This is the single most important thing you can do to

harden this app beyond its built-in checks — see [SECURITY.md](#) for more detail.

Governance & Risk Posture

Alongside the technical controls in Chapters 7–8, this project now documents *who is responsible for what*, and *how a security-relevant change gets made*, in writing — `docs/GOVERNANCE.md` and `docs/security-changelog.md`. This chapter summarizes that document for a general audience; it is not a substitute for reading it.

Ownership

This is a solo-maintainer project. One person is explicitly and solely responsible for its data-handling and security decisions — what it connects to, what SQL is allowed to run, what limits are set, and whether any exception to those is granted. There is no separate review board or second approver, and the documentation says so plainly rather than implying otherwise.

Data classification policy — honestly, a work in progress

The governance documentation defines an intended three-tier policy for classifying columns as **public**, **internal**, or **restricted**. As of this writing, that classification system is **not yet implemented as an enforced control** — there is no per-column sensitivity file and nothing in the app blocks or redacts a sensitive column automatically. This is stated here deliberately, not glossed over: two things reduce related risk today as a side effect (a cap on how many distinct values a column needs before it's eligible for sampling into a prompt, and the table-notes file's plain-language descriptions) but neither is a real classification or enforcement mechanism. This gap is tracked openly rather than assumed away.

Change control

Any change to the SQL validator's allowlist, rate limits, or cost-estimation thresholds is meant to be a deliberate, dated entry in `docs/security-changelog.md` — separate from ordinary feature commit history — so "when and why did a limit change" can be answered without reconstructing it from scratch.

Exception process

Sometimes a control needs to be knowingly, temporarily relaxed — for example, raising a rate limit for a demo. The documented process: state it before making the change, log it with a reason and a review/expiry date, and revert or explicitly re-review it by that date, rather than let it quietly persist.

Review cadence

Proportionate to a solo-maintainer project: re-run the accuracy/eval checks after any change to the prompt, configuration, or AI model, and review recent activity on a regular cadence during active development — not an enterprise-scale schedule, a realistic one.

Testing & Quality Checks

Automated unit tests

The mocked test suite runs without a real database or Ollama connection, covering the SQL validator, connection handling, schema introspection/retrieval, the agent's node logic, follow-up classification, rate limiting, and —  — a dedicated adversarial-input test module.

```
pytest
# or: .\tasks.ps1 test
```

Linting & formatting

```
ruff check . && black --check . && mypy .
# or: .\tasks.ps1 lint
```

Accuracy evaluation harness

Separate from the unit-test suite, an evaluation harness runs a growable set of real natural-language questions against a live database and a live Ollama model:

- **Minimum row count** — did the query return a plausible amount of data?
- **Readable result** — is the first result column something a person can read, not a bare internal ID?
- **Correct tables used** — does the executed SQL actually reference every table the question logically requires?
-  **Multi-turn follow-up sequences** — a real two-or-more-question conversation, checking that a follow-up is correctly classified and correctly resolved against the prior turn.
-  **Adversarial cases** — a permanent regression category of injection attempts, homoglyph-obfuscated attacks, off-topic requests, and malformed input, where success means the app *declines cleanly*, not that it answers well.
-  **Cost-estimation cases** — deliberately broad questions with no natural row limit, checking that a "moderate" cost still runs (with a notice) and a "high" cost never executes at all.

```
python scripts\run_eval.py
```

Known Limitations

- **Accuracy depends heavily on the local model.** The default model handles straightforward aggregation/filter/join questions well but is noticeably weaker on window-function-heavy or deeply nested subquery questions, and can occasionally pick the wrong column even on questions it usually gets right (e.g. confusing a product's own name with its category) — the same question can succeed on one attempt and return an empty, technically-valid result on another. This is why the retry timeline and the accuracy eval harness both exist: to make that variability visible and measurable rather than silent.
- **Local inference is slower than a hosted API**, especially across several retry attempts on a hard question.
- **Schema retrieval can still miss a required table** on schemas with long, multi-hop hierarchies if more than one connecting hop is missing at once.
- **Single-user, local-dev oriented.** Rate limiting is a basic, in-memory, single-process safeguard sized for one local user — not a substitute for real multi-tenant rate limiting.
-  **Column-level sensitivity classification is not yet built** — see Chapter 9.
-  **Input screening is a first filter, not a proof** — see Chapter 7's callout on what actually bounds the risk.

Troubleshooting

Symptom	Likely cause & fix
Red "Database Connection Required" screen on launch	The database couldn't be reached. Run <code>python scripts\test_db_connection.py</code> for a detailed diagnostic.
Connection fails with an authentication error	Check <code>DB_USER</code> and <code>DB_PASSWORD</code> .
"Driver missing" error, SQL Server	The Microsoft ODBC Driver for SQL Server must be installed as a system package.
App seems to hang on the first question	Ollama may still be loading the model into memory for the first call — normal, one-time.
NEW "Your question was declined" message	The question tripped the input screener (Chapter 7) — check the specific reason shown. If it's a legitimate database question that got flagged, try rephrasing plainly.
NEW "You're asking questions faster than allowed" message	You've hit the per-session rate limit — wait under a minute and try again, or raise <code>QUESTION_RATE_LIMIT_PER_MINUTE</code> in <code>.env</code> if this is genuinely too strict for your use.
NEW Query says "too expensive to run" and fails after retries	The estimated result size exceeded <code>COST_HIGH_ROW_THRESHOLD</code> . Narrow the question (a filter, a date range, "top N") or raise the threshold in <code>.env</code> if the broad query is genuinely intentional.
PowerShell won't run <code>Activate.ps1</code>	Check <code>Get-ExecutionPolicy</code> as yourself (not admin) and set it to <code>RemoteSigned</code> for the CurrentUser scope.

Frequently Asked Questions

Can this ever change or delete my data?

No. Every query is restricted to read-only lookups before it's allowed to run.

What happens if I ask something unrelated to my database?

It's declined immediately with a clear message, before any AI call is made — see Chapter 7.

Is there a limit to how many questions I can ask?

Yes, a generous default (10 per minute per session) meant to catch accidental loops or abuse, not normal use. It resets automatically.

What's the "AI insight" line above my results?

An optional, one-sentence plain-English observation about the result (e.g. the top category and its share of the total). Every number in it is checked against your actual result data before it's shown — if it can't be verified, it's simply not shown, rather than risking a made-up figure. Turn it off any time via the sidebar's "Generate AI insight" toggle.

Can I ask a follow-up without repeating myself?

Yes — see Chapter 5. If it's ambiguous, the app asks you to clarify rather than guessing.

Why does it sometimes take a few extra seconds?

Usually the retry loop working, or a cost check running before an unusually broad query — both make you wait a little longer in exchange for a more correct or safer answer.

Which databases are supported?

PostgreSQL, MySQL, SQL Server, and Oracle, selected via `DB_TYPE` in `.env`.

Does any of my data or my questions get sent anywhere?

No. Everything — the AI model, the schema search index, the database connection — runs locally.

Glossary

Term	Plain-language meaning
LLM	Large Language Model — the AI model that turns your question into SQL. Runs locally here via Ollama.
Ollama	The local software that runs the AI model on your own machine, no internet required.
LangGraph	The library used to build the app's explicit, step-by-step decision flow, instead of a free-form AI agent.
ChromaDB	A local search index used to find which database tables are relevant to a given question.
sqlglot	The library that parses generated SQL into its real structure so it can be safety-checked.
Allowlist	A security approach that only accepts things explicitly known to be safe, rather than trying to block everything known to be dangerous.
Surrogate key	An internal ID column used to link tables together, with no business meaning of its own — hidden from the default results view.
 Prompt injection	An attempt to manipulate an AI model by phrasing input as if it were a new instruction, rather than data to process (e.g. "ignore your previous instructions and...").
 Homoglyph	A character from a different alphabet that looks nearly identical to a familiar one (e.g. a Cyrillic "o" that looks like a Latin "o") — sometimes used to disguise text from simple pattern matching.
 Rate limiting	Capping how many actions (questions, AI calls) can happen in a given time window, to prevent accidental or deliberate overload.
 Cost estimation	Asking the database how big a query's result is likely to be, without actually running it, so an extremely expensive query can be caught before it runs.
 Follow-up question	A question that only makes sense in light of a previous one in the same session (e.g. "and how many of those?").
 Grounded insight	An AI-generated summary sentence that's automatically checked against the real result data before being shown, so it can't state an unsupported number.
 Governance	The written policy for who's responsible for a project's security/data decisions and how changes to its safety-relevant settings are tracked over time.

Further Reading

This document is the comprehensive, user-facing reference. A few other files in the repository go deeper on specific areas:

File	What's in it
README.md	Quick-start overview and setup, aimed at a first-time visitor to the repository.
docs/ARCHITECTURE.md	The full code-level technical walkthrough: the LangGraph node design, retry/self-correction logic, and the schema-retrieval pipeline internals.
SECURITY.md	This project's full security posture, including the input-safety, rate-limiting, and cost-estimation layers, and how to report a vulnerability.
 docs/GOVERNANCE.md	Ownership, data-classification policy, change control, and the exception process for security-relevant settings.
 docs/security-changelog.md	The dated log of changes to the SQL validator, rate limits, and cost thresholds.
CONTRIBUTING.md	Developer setup, coding standards, and how to add a new evaluation question.

Text-to-SQL Dashboard — Documentation. Reflects the project as of its current state in this repository.